construct the schema by hand and use the `Optional` Python type hint. Implementing PATCH means you have to write more serializer code, so it needs to be worth it. The custom schema notwithstanding, PATCH code is otherwise identical to a PUT: loop through the fields sent and update the object, saving it when you're done.

Choosing between PUT and PATCH is based on how you're passing objects around. For an SPA or a mobile app, there is a good chance the entire object gets sent to the browser or device, and the browser or device doesn't track changes at the field level. In this case, it is simplest to send all the information on the client side to the server through a PUT. Alternatively, for models with many fields or for data-intensive fields, like attachments, you might use PATCH. It is up to the client device to know what fields got edited in this case.

Last, but not least, is DELETE. OK, technically it is least, as in the least likely thing you'll need to implement. I've never counted, but I suspect less than a quarter of the APIs I've built have implemented delete capabilities, but that will depend on what you're using the API for. If your API is to make data available programtically (like an RSS feed), it will likely be read only and not need DELETE. If your API is the interface to a mobile front end, you'll likely need the whole range of CRUD.

To delete an object, you need to find it. Like with PUT and PATCH, the URL for DELETE contains an object ID. What the body of a delete call's response should contain is somewhat debated. The two schools of thought are (a) return the deleted object or (b) return some meta-information expressing success or failure. I lean toward the latter, but if you want to give the user undo or re-save capabilities, returning the object might be your preferred approach. The following listing contains code to delete a venue. Nothing in it should be a surprise at this point.

Listing 11.15 A delete endpoint for venues

```
# RiffMates/bands/api.py
...

@router.delete("/venue/{venue_id}/", auth=api_key)
def delete_venue(request, venue_id):
    venue = get_object_or_404(Venue, id=venue_id)
    venue.delete()

    return {"success": True}
```

Use the Ninja docs interface to try out your new delete endpoint. Once you're done, run a GET on the newly deleted ID. On the server side, this causes a 404, as the lookup in `get_object_or_404()` throws an error. It is worth trying out to see how Ninja handles a `404`. The HTTP status code gets set, of course, but it also automatically sends a JSON dictionary containing a key named *details*, whose value is the error string from the exception. This can be useful in a user interface: you can show the error in a pop-up dialog.